

TCN



진행 상태

진행 중

Abstract

보통 우리가 생각하기에 시퀀스 데이터를 다룰 때는 순환 신경망(RNN, LSTM 등)을 사용한다.

하지만 최근 연구에서는 CNN 기반 아키텍처가 오디오 합성이나 기계 번역 같은 task에서 더 우수한 성능을 보여준다.

이에 대해서 어떤 모델을 사용할지 알아보기 위해

순환 신경망을 벤치마크할 때 흔히 사용되는 표준 task를 이용하여

일반적인 합성곱 아키텍처와 순환 신경망 아키텍처를 비교 평가한다.

결과적으로 단순한 합성곱 아키텍처가 순환 신경망보다 성능이 높게 나왔으며, 더 긴 장기 기억력을 가지고 있었다.

Introduction

CNN 아키텍처가 시퀀스 모델링 task에서 SOTA의 성능을 낸다는 연구가 등장하면서

- 그저 특정 분야에서만 CNN 아키텍처가 성공을 거둔 것인가?
- 만약 더 넓은 분야에서도 효과적이라면 시퀀스 데이터와 순환 신경망 사이의 관계를 다시 생각해야하나?

라는 의문이 들 수 있다.

이에 대한 의문을 해결하기 위해

- polyphonic music modeling

- word and character-level language modeling
- synthetic stress tests

위와 같은 과제들에 대해 합성곱, 순환 아키텍처들을 평가한다.

합성곱 네트워크는 TCN이라는 모든 task에 동일하게 적용하는 일반적인 아키텍처를 사용한다.

지금까지 순환 신경망을 평가하기 위해 사용한 벤치마크에 대해서도 TCN은 뛰어난 성능을 보여주었다.

이러한 실험결과는 CNN 아키텍처의 성공이 단순히 특정 분야에 국한된 것이 아니라는 것을 보여준다.

시퀀스 모델링을 위해서 순환 신경망을 사용해야한다는 생각을 재고해야할 것이며, TCN이 적절한 출발점이 될 수 있다.

Background

시퀀스 데이터를 다루는 모델로 RNN은 아주 큰 인기를 얻었다. 다만 학습 난이도가 높으므로 조금 더 구조를 정교하게 한 LSTM과 GRU가 주로 사용되었다.

최근 연구에서는 CNN과 순환 신경망들을 결합하는 시도를 해왔는데 그 예시로

Convolutional LSTM : LSTM의 완전연결 계층을 합성곱 계층으로 대체

Quasi-RNN : 합성곱 계층과 단순한 순환 계층을 교차하여 결합

Dilated RNN : 순환 구조에 dilation을 도입

를 들 수 있다.

이러한 결합들은 두 아키텍처의 장점을 함께 활용하려는 시도로서 가능성을 보여주지만

합성곱 접근법과 순환 접근법을 시퀀스 모델링 **자체**에서 동일하게 철저히 비교한 연구는 많지 않다.

따라서 이 논문의 목표는 RNN 변형들의 벤치마크 과제로 널리 사용되는 전형적인 시퀀스 모델링 과제들에서 일반적인 합성곱 아키텍처와 순환 아키텍처를 비교하는 것이다.

Temporal Convolutional Networks

TCN은 특정 모델의 이름이 아니라 시계열 합성곱 신경망이라는 아키텍처의 계열 이름이다. 두 특징을 가진다.

1. 미래의 정보가 과거에 영향을 주지 않는다.
2. 임의 길이의 입력 시퀀스를 받아 동일한 길이의 출력 시퀀스로 매핑할 수 있다.

또한 긴 과거의 데이터를 다룰 수 있게 하며, residual 레이어, dilated convolutions을 결합한다.

Sequence Modeling

모델의 구조를 설명하기 전에 시퀀스 모델링이라는 과제에 대해서 알아보자

$$\hat{y}_0, \dots, \hat{y}_T = f(x_0, \dots, x_T)$$

입력 시퀀스 x 에 대해서 출력 시퀀스 y 로 매핑하는 것이다.

단, 중요한 점은 미래의 정보를 볼 수 없는 causal constraint를 지켜야 한다는 것.

즉, y_t 는 오직 $x_0, \dots, x_t$ 의 정보에만 의존해야 하며, 미래 입력 x_{t+1} 이후는 의존할 수 없다는 것이다.

물론 이는 sequence-to-sequence과 같은 도메인은 설명하지는 못한다. 하지만 아이디어 자체는 확장시킬 수 있다.

Causal Convolutions

상기했던 TCN의 두 특징을 만족시키기 위해서

1. TCN은 1D FCN을 사용한다.
 - 각 은닉층 길이 = 입력층 길이
 - 다음 레이어에서도 길이가 유지될 수 있도록 ($kernelsize - 1$) 길이의 zero padding을 추가한다.

2. TCN은 causal convolution을 사용한다.

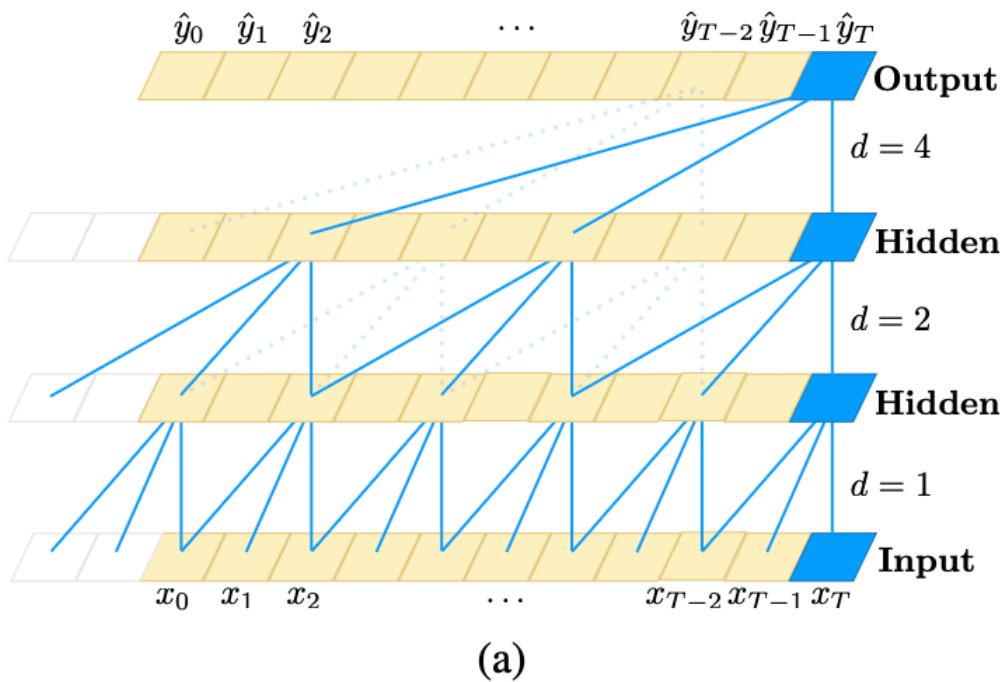
- t 시점의 출력은 t 시점 및 그 이전의 요소들과만 합성곱 연산을 한다

다만 이 두 기법을 사용한 네트워크는 강력한 장기 기억력을 위해서 매우 깊은 네트워크와 필터가 필요하다. 이를 위해 현대의 합성곱 아키텍처 기법들을 TCN에 통합하여 해결해보자

Dilated Convolutions

Causal Convolutions 기법을 사용하면 레이어 깊이에 비례한 정도의 과거 정보를 활용할 수 있다. 매우 긴 과거까지 활용하려면 아주 많은 자원이 필요할 것이다.

그래서 Dilated Convolutions을 사용한다.



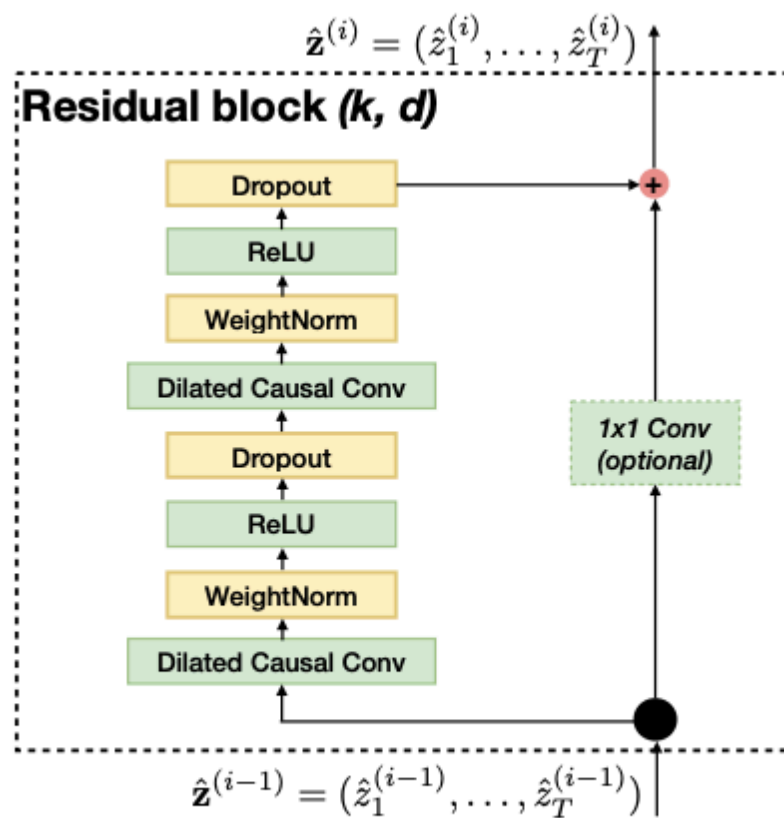
$$F(s) = (x *_d f)(s) = \sum_{i=0}^{k-1} f(i) \cdot x_{s-d \cdot i}$$

- d : 팽창 계수(dilation factor),
- k : 필터 크기(filter size),
- $s - d \cdot i$: 과거 방향

$d = 1$ 일 때는 일반 합성곱과 동일하며 이 값이 커질수록 일부 시점의 입력은 건너뛰며 점점 더 넓은 범위의 입력을 받을 수 있다.

일반적으로 i 가 레이어의 깊이라 할 때 $d = 2^i$ 로 점차 증가시킨다. 이렇게 하면 짧은 필터를 이용하더라도 보다 적은 레이어를 통해 더 많은 입력을 수용할 수 있다.

Residual Connections



(b)

TCN의 수용영역은 깊이(n), 필터 크기(k), 팽창 계수(d)에 의존하는데 더 큰 수용영역을 위해서 더 깊은 네트워크를 만들어야한다. 이 때 더 안정적인 학습을 위해 단순 합성곱 대신 residual module을 사용한다. 이때 사용하는 Dropout은 spatial dropout으로 매 훈련 단계마다 전체 채널이 0으로 설정된다.

TCN과 같은 구조에서는 입출력의 크기가 다를 수 있다. 따라서 1*1 Conv를 이용해 입출력의 크기를 동일하게 설정하였다.

Discussion

장점

- 병렬성(Parallelism)

RNN은 이전 시점이 끝나야 그 이후 시점을 예측할 수 있지만 TCN은 공유된 필터를 사용하기 때문에 병렬적으로 처리할 수 있다.

- 유연한 수용 영역 크기(Flexible receptive field size)

Dilation과 같은 방법으로 수용 영역의 크기를 조절할 수 있다. 메모리 크기에 대해서 더 유연하게 대처할 수 있고 다양한 도메인에도 적용할 수 있다.

- 안정적인 기울기 (Stable gradients)

TCN은 역전파 경로가 시간 축과 다르기 때문에 기울기 소실이나 폭발 문제가 적다.

- 낮은 학습 메모리 요구량 (Low memory requirement)

RNN은 게이트 연산을 위한 중간 결과를 저장해야 하므로 메모리를 많이 사용한다. 반면 TCN은 필터를 공유하고 역전파가 깊이에만 의존하므로 메모리 사용이 적다.

- 가변 길이 입력 처리 (Variable length inputs)

TCN은 1D 합성곱 커널을 슬라이딩하여 임의의 길이의 입력을 처리할 수 있다. 따라서 순차 데이터 처리에서 RNN을 대체할 수 있다.

단점

- 평가 시 데이터 저장 요구량 (Data storage during evaluation)

RNN은 평가 단계에서 hidden state만 유지하면 되고, 현재 입력 x_t 만 받아서 예측을 생성한다. 즉, 전체 과거 시퀀스는 h_t 에 요약되므로 원래 입력 시퀀스를 보관할 필요가 없다.

반면 TCN은 effective history length만큼의 원본 시퀀스를 입력으로 받아야 하므로, 평가 단계에서 더 많은 메모리를 요구한다.

- 도메인 전이 시 파라미터 조정 필요성 (Potential parameter change for transfer of domain)

도메인마다 예측을 위해 필요한 기억 범위가 다를 수 있다. 따라서 작은 값의 k, d 로도 충분한 도메인에서 학습한 TCN을, 훨씬 긴 기억이 필요한 도메인으로 옮기면 수용 영역이 충분히 크지 않아 성능이 저하될 수 있다.

Experiment

Synthetic Stress Tests

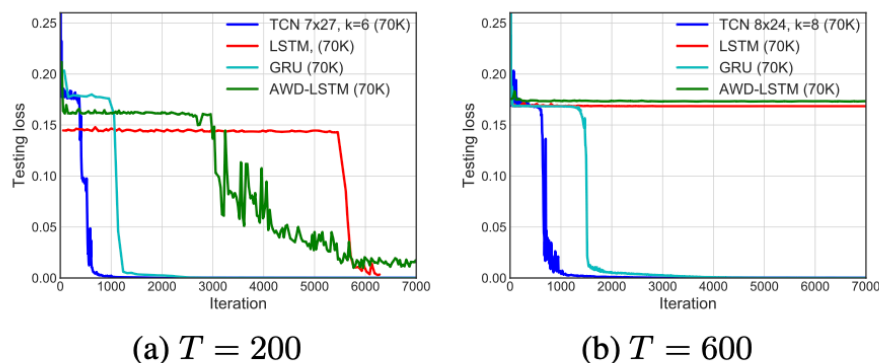


Figure 2. Results on the adding problem for different sequence lengths T . TCNs outperform recurrent architectures.

Adding Problem ($T=200, 600$)

- **TCN:** 빠르게 $MSE \approx 0$ 도달 $\rightarrow$ 사실상 완벽한 학습.
- **GRU:** 좋은 성능, 하지만 TCN보다 수렴 속도 느림.
- **LSTM / RNN:** 성능 크게 떨어짐.

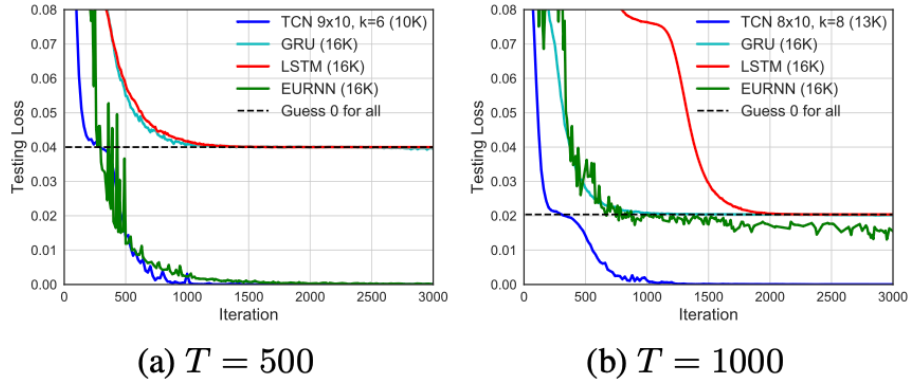


Figure 4. Result on the copy memory task for different sequence lengths T . TCNs outperform recurrent architectures.

Copy Memory Task

- **TCN:** 빠른 정답 수렴.
- **LSTM / GRU:** "모두 0 예측" 수준에서 수렴 → 학습 실패.
- **EURNN:** $T=500$ 에서는 TCN과 비슷한 성능.
- **$T=1000$ 이상:** TCN이 EURNN보다 손실과 수렴 속도 모두 우월.

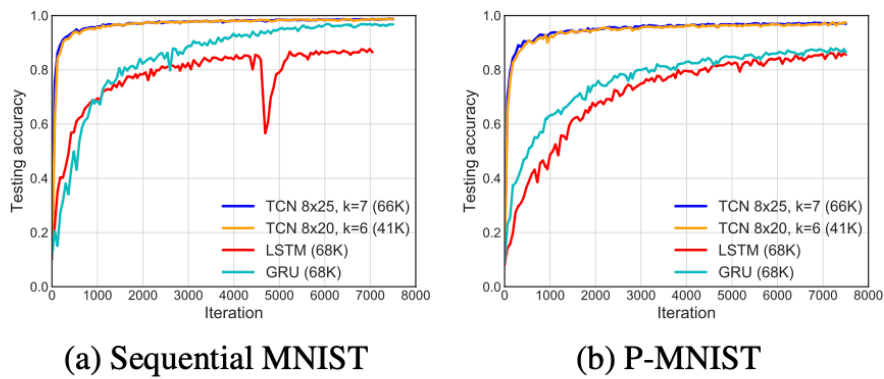


Figure 3. Results on Sequential MNIST and P-MNIST. TCNs outperform recurrent architectures.

Sequential MNIST / P-MNIST (10 epochs, ~70K 파라미터)

- **TCN:** 빠른 수렴 + 높은 최종 정확도.
- **P-MNIST:** TCN이 기존 RNN 기반 SOTA(95.9%) 초과.
- **RNN 계열:** TCN 대비 수렴 느리고 정확도 낮음.

Polyphonic Music and Language Modeling

다성음악 모델링 (Nottingham, JSB Chorales)

- **TCN**: 튜닝 거의 없이도 RNN보다 훨씬 우수.
- HF-RNN, Diagonal RNN보다도 성능 우수.
- 하지만 **Deep Belief Net LSTM**이 가장 좋은 성능 (소규모 데이터셋에서 정규화/생성 모델링 기법 효과).

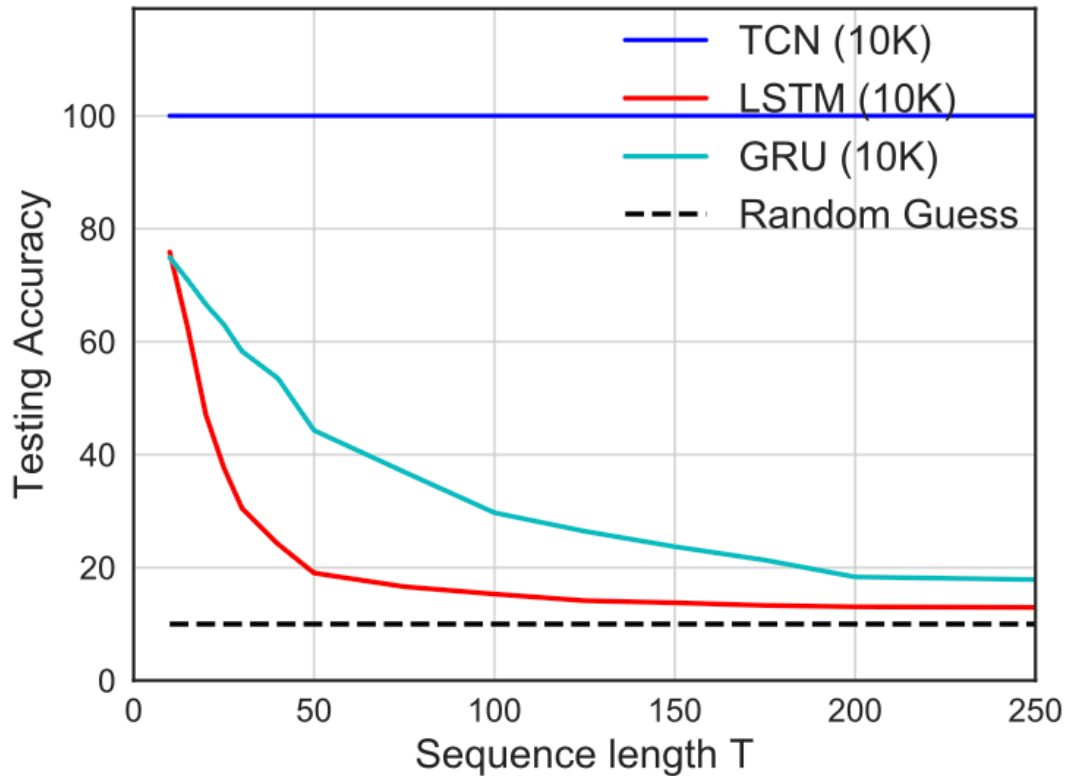
단어 단위 언어 모델링 (PTB, Wikitext-103, LAMBADA)

- **PTB (소규모)**: 최적화된 LSTM > TCN > GRU, RNN.
- **대규모 (Wikitext-103, LAMBADA)**: TCN이 LSTM을 능가, perplexity 크게 낮춤.
- 결론: 작은 데이터셋에서는 LSTM이 강점, 큰 데이터셋에서는 TCN이 더 효율적.

문자 단위 언어 모델링 (PTB, text8)

- **TCN > 정규화된 LSTM, GRU, Norm-stabilized LSTM.**
- 단, 특수화 아키텍처들은 TCN보다 더 나은 성능 가능.

Memory Size of TCN and RNNs



Copy Memory Task (모델 크기: 10K 파라미터)

- **TCN:** 시퀀스 길이에 상관없이 100% 정확도 도달.
- **LSTM:** $T < 50$ 에서 정확도 $< 20\%$.
- **GRU:** $T < 200$ 에서 정확도 $< 20\%$.
- → TCN이 훨씬 더 긴 시퀀스 의존성을 학습·유지 가능.

LAMBADA Task (실제 데이터, 문맥 활용 능력 평가)

- **TCN:** 작은 네트워크 + 거의 무튜닝 상태에서도 LSTM, RNN보다 월등히 낮은 perplexity.
- **LSTM/RNN:** 긴 문맥 처리 능력이 부족해 성능 열세.
- **SOTA:** 추가 메모리 메커니즘(RAM, external memory 등)을 활용한 모델이 더 좋은 성능을 기록하지만, 이는 구조 자체 비교와는 별개.

Conclusion

위에서 진행했던 실험을 통해 TCN 모델이 LSTM과 GRU와 같은 순환 신경망 모델보다 좋은 성능을 보인 것을 확인할 수 있었다.

메모리적인 측면에서도 이론상으론 RNN은 무한한 메모리를 가진듯 보이지만 실질적으로는 불가능했고 TCN이 더 긴 메모리를 가지고 있음을 알 수 있었다.

LSTM은 이를 정규화하고 최적화하기 위한 수많은 고급 기법들이 등장했다. 그에 반해 TCN은 아직 이러한 연구가 활발히 이루어지지 않고 있다.

이 논문의 연구 결과로 팽창 합성곱(dilated convolution)이나 잔차 연결(residual connection) 같은 요소가 도입되어 시퀀스 모델링 과제에 더 효과적인 성능을 보임에 따라 이에 대한 연구의 필요성을 보여준다.